

code is designed for a real-time EEG (Electroencephalography) based stress detection system. It involves receiving EEG data, processing it, predicting stress levels using a pre-trained model, and displaying the results on a graphical user interface (GUI).

1. Imports and Global Variables

The code imports necessary libraries and sets up global variables such as IP address, port, file paths, and model path.

2. Loading the Trained Model

- **load_model(path)**: This function loads a pre-trained TensorFlow model from the specified path. The model is designed for EEG data and has three dense layers with dropout for regularization.

3. Preprocessing Data

- **preprocess_data(features)**: This function scales the features using `StandardScaler` to normalize the data. It ensures that the number of features matches the model's expected input size (2548). If there are fewer features, it pads with zeros; if there are more, it truncates the excess.

4. Writing File Header

- **writeFileHeader()**: Writes the header for the CSV file where EEG data will be stored. The header includes timestamps and EEG channel labels.

5. Handling Incoming EEG Data

- **eeg_handler(address, *args)**: Handles incoming EEG data, formats it with a timestamp, and writes it to the CSV file.

6. Streaming EEG Data

- **stream_eeg()**: Listens on a UDP socket for incoming EEG data, decodes it, and passes it to `eeg_handler`

7. Displaying EEG Values

- **display_eeg()**: Continuously prints the current EEG values to the console. This is mainly for debugging purposes.

8. Real-time Prediction

- **real_time_prediction()**: Continuously reads EEG data from the CSV file in batches, processes it, and makes predictions using the loaded model. The predictions are then displayed in the GUI.

9. Updating GUI

- **update_gui(percentages)**: Updates the GUI labels with the latest prediction percentages. If the stress level is above 80%, the stress label is highlighted in red.

10. Main Execution Block

- The main block initializes the CSV file header and starts three threads:
 - **stream_thread**: For streaming EEG data.
 - **display_thread**: For displaying EEG values in the console.
 - **prediction_thread**: For making real-time predictions.
- It also sets up the GUI using Tkinter:
 - **root**: The main Tkinter window.
 - **GUI Elements**: Labels and buttons for displaying results and interacting with the system.

GUI Components

- **Title Label**: Displays the title of the application.
- **Stream Button**: Placeholder for a button to start streaming data (implementation not shown).
- **Prediction Labels**: Display the prediction percentages for stress, calm, and good mood.
- **Note Label**: Provides a note to the user about stress level indication.

The overall flow is as follows:

1. The system listens for EEG data on a UDP port.
2. Incoming data is formatted and written to a CSV file.
3. Periodically, the system reads the CSV file, preprocesses the data, and uses the model to predict stress levels.
4. The results are displayed in a GUI, updating in real-time.

